

Scott's Toolkit v5

Context Engineering for Claude Code

A Technical Briefing for Gary Fenn, Integrator
Prepared by Scott O'Dell, Head of Sales
March 2026

What Is the Toolkit?

A configuration layer on top of Claude Code. It controls how Claude behaves during development: what rules it follows, what checks it runs, and what quality gates it must pass before code ships. It remembers past mistakes and applies those lessons to every future project.

The one-sentence version

The toolkit makes Claude Code remember what worked, enforce what matters, and get smarter with every project.

The Problem It Solves

No memory. Each Claude session starts fresh. It will not remember that SurrealDB v3 silently drops undefined fields, or that the last three projects had timezone bugs. The toolkit captures those lessons and enforces them going forward.

Unreliable instructions. Rules in a CLAUDE.md file are followed about 80% of the time. The toolkit uses hooks (automated scripts) to make critical steps 100% reliable. A hook physically blocks marking a phase complete unless a full quality review has been done.

No gates. Without enforcement, code review and compliance checks get skipped when sessions run long. The toolkit makes these mandatory.

How It Works

Four systems, each owning a lane:

System	Owns	What It Does
Scott-Toolkit	Context engineering	Session management, templates, stack enforcement, learning capture, orchestration
GSD (plugin)	Project management	Phase planning, execution, verification, task tracking
Superpowers (plugin)	Dev methodology	TDD, code review, git worktrees, branch management
Impeccable (plugin)	UI/UX quality	Design critique, visual polish, accessibility audits

The build loop runs in order: worktree (isolate) > plan (structure) > execute with TDD (build) > code review (two-pass) > verify (test against acceptance criteria) > design review (UI only) > finish branch (merge).

Hooks: The Enforcement Layer

16 automated hooks. Unlike instructions Claude might forget, these execute every time.

Hook	What It Enforces
guard-phase-completion	Blocks phase completion unless full quality review (tests, stack audit, code review, retro) is done
guard-git-push	Prevents pushes to protected branches
guard-npm-install	Enforces pnpm. Catches stack drift
auto-format	Runs Prettier on every edit
context-reminders	Warns at 60min / 100 tool calls to prevent degraded output
pre-compact	Snapshots session state before compaction
offload-large-output	Moves large output to files instead of filling context window

Stack Enforcement

Each technology has a check file (JSON) listing patterns to catch. These run against every changed file during phase closeout. Check files currently cover SurrealDB v3, Nuxt 4, Tailwind CSS v4, Bun, and Hono.

Real example

Two code-reading passes missed 3 bugs (missing tenant_id, missing delivery_address, wrong schema load order). A live SurrealDB audit caught all 3 in minutes. Stack enforcement was built to eliminate this class of failure.

What Makes It Unique

Decoupled from its plugins

The toolkit uses abstract operation names ("plan_phase", "tdd", "code_review") mapped to real commands through interfaces.json. If GSD renames /gsd:plan-phase to /gsd:plan, one line changes. Every workflow keeps working. GSD and Superpowers can update, rename, or restructure without breaking the toolkit. 17 operations currently mapped across 3 tools.

The Learning Loop

The toolkit gets smarter with every project through a structured cycle:

Step	What Happens	Output
1. Build	Build features using the toolkit	Working code
2. Closeout	Mandatory review: tests, stack audit, code review, reflection	Error logs, success logs, tagged lessons
3. Promote	[stack]-tagged lessons become check candidates	New automated checks
4. Apply	New checks run on every future project	Better enforcement

A bug caught on one project becomes a check that prevents the same bug on every future project. The toolkit accumulates institutional knowledge over time.

Research Backing

The toolkit aligns with the current consensus in context engineering. Key principles:

Hooks over instructions. CLAUDE.md rules are ~80% reliable. Hooks are 100%. Anthropic's engineering team and independent practitioners agree: critical actions belong in hooks.

Anthropic, "Effective Harnesses" (2026); HumanLayer, "Harness Engineering" (2026)

Lean context windows. Performance degrades past ~40% context utilization. The toolkit keeps rules to 173 lines and uses deferred loading and file offloading.

Phil Schmid, "Agent Harness Importance" (2026); LangChain Terminal Bench 2.0

Structured execution. Research > plan > execute > verify outperforms ad-hoc coding across all benchmarks. LangChain improved 52.8% to 66.5% by changing only the harness, not the model.

LangChain Terminal Bench 2.0 (2026); Anthropic harness research

Feedback loops. Running tests and linters after every change improves output quality 2-3x. The toolkit's auto-format hook, stack checks, and closeout gate serve this principle.

Martin Fowler, "Context Engineering for Coding Agents" (2026); Anthropic docs

How It Improves Over Time

Five compounding mechanisms:

Errors become prevention. Mistakes captured during closeout include root cause and prevention steps. Tagged [stack] lessons become automated checks.

Successes become standards. Patterns that work are captured and promoted to skills or rules during monthly maintenance.

Retros drive updates. Each retro produces a "Toolkit Updates Needed" section with specific file changes.

Instincts are captured. A hook automatically captures patterns before context compaction. These are reviewed and promoted or discarded monthly.

Checks get sharper. Audit data aggregates across all projects. Pass/fail rates per technology identify where checks need tuning.

The compounding effect

Project 1: Base checks catch common patterns.

Project 5: Base checks plus lessons from 4 projects catch stack-specific issues.

Project 15: Comprehensive check library built from real mistakes across multiple codebases.
This advantage is built through use. It cannot be replicated by starting over.

What This Means for You

When Scott hands off a project built with the toolkit, the code has been through:

- TDD on every feature (tests written before implementation)
- Two-pass code review with fix verification
- Stack compliance checks against technology-specific check files
- A handoff workflow that produces documentation, architecture notes, tech debt inventory, and a production-readiness checklist
- A stack-lock.json that locks all technology versions
- Known issues and tech debt documented honestly

The toolkit does not change your production workflow. It changes the quality of what arrives at your door.

Questions? Reach Scott at scott@advosy.com.